

Detailed Notes

Chapter 5 — Pointers

Topics Covered

- Introduction to Pointers
 - What Are Pointers?
 - Pointer Variables
- Declaration and Base Type
 - The Pointer Operators
 - & (Address-of Operator)
 - * (Dereference / At-address Operator)
 - Pointer Expressions
 - Pointer Assignments
- Pointer Conversions (void *, explicit casts)
- Pointer Arithmetic (increment, decrement, add, subtract)
 - Pointer Comparisons (stack example)
 - Pointers and Arrays
 - Array Indexing vs Pointer Arithmetic
 - Arrays of Pointers
 - Pointers to Functions
- Multiple Indirection (Pointers to Pointers)
 - Initializing Pointers
 - Null Pointer Convention
 - Null Pointer as Sentinel
 - char * Pointers and String Constants
- Dynamic Allocation (malloc and free)
 - Dynamically Allocated Arrays
 - restrict-Qualified Pointers (C99)
 - Problems with Pointers

CHAPTER 5 — POINTERS

The correct understanding and use of pointers is crucial to successful C programming. Pointers are one of the strongest but also one of the most dangerous features in C.

Reason Pointers Matter	Explanation
Modify calling arguments	Pointers provide the means by which functions can modify their calling arguments (call by reference)
Support dynamic allocation	Pointers are required for malloc(), calloc(), and all dynamic memory management
Improve efficiency	Pointer arithmetic is often faster than array indexing; certain routines run faster using pointers
Support dynamic data structures	Pointers enable linked lists, binary trees, stacks, queues, and other dynamic structures

NOTE *A pointer containing an invalid value can cause your program to crash. It is easy to use pointers incorrectly, causing bugs that are very difficult to find. Because of their importance and their potential for abuse, this chapter examines the subject of pointers in detail.*

WHAT ARE POINTERS?

A pointer is a variable that holds a memory address. This address is the location of another object (typically another variable) in memory. If one variable contains the address of another variable, the first variable is said to point to the second.

Term	Definition
Pointer	A variable that holds a memory address
Points to	If variable A contains the address of variable B, then A points to B
Memory address	The computer's internal location number for a variable or object
Object	The item stored at the address a pointer holds (typically another variable)

Conceptual diagram — a pointer variable P holding address 2000, where the value 100 is stored:

Variable / Location	Contents	Role
P (pointer variable)	2000	Holds the address of count

Variable / Location	Contents	Role
Memory location 2000 (count)	100	The actual value — the object P points to

POINTER VARIABLES

If a variable is going to be a pointer, it must be declared as such. A pointer declaration consists of a base type, an *, and the variable name.

General form:

```
type *name;
```

Component	Meaning
type	The base type of the pointer — may be any valid C data type
*	Indicates that this variable is a pointer
name	The name of the pointer variable

Examples of pointer declarations:

```
int  *ip; /* pointer to an integer */
double *dp; /* pointer to a double */
char  *cp; /* pointer to a character */
float *fp; /* pointer to a float  */
```

Key rules about pointer base types:

- The base type defines the type of object to which the pointer will point
- Technically, any type of pointer can point anywhere in memory
- However, ALL pointer operations are performed relative to the pointer's base type
- When you declare a pointer as `int *`, the compiler assumes that any address it holds points to an integer — whether it actually does or not
- An `int *` pointer always 'thinks' it points to an `int` object, no matter what that memory actually contains
- Therefore, you must make sure the pointer's type is compatible with the type of object to which you want to point

THE POINTER OPERATORS

There are two pointer operators: `&` and `*`. Both are unary operators (they require only one operand).

Operator	Name	Action	Think of it as...
&	Address-of	Returns the memory address of its operand	"the address of"
*	Dereference / At-address	Returns the value located at the address that follows it	"at address"

The & (Address-of) Operator

The & operator returns the memory address of its operand:

```
m = &count;
/* Verbalized: 'm receives the address of count' */
/* If count is stored at location 2000, then m now contains 2000 */
/* This has nothing to do with the VALUE of count */
```

Step-by-step illustration:

Assumption / Step	Result
count is stored at memory location 2000	count occupies address 2000
count currently has the value 100	value at location 2000 = 100
Execute: m = &count;	m now contains 2000 (the address, not the value)

The * (Dereference) Operator

The * operator is the complement of &. It returns the value located at the address that follows it:

```
q = *m;
/* Verbalized: 'q receives the value at address m' */
/* Since m = 2000 and count = 100 is stored at 2000, q gets 100 */
```

Complete example using both operators together:

```
#include <stdio.h>
int main(void)
{
    int count = 100;
    int *m;
    int q;
```

```

m = &count; /* m = address of count (e.g., 2000) */
q = *m;     /* q = value at address m = 100 */

printf("count = %d\n", count); /* 100 */
printf("m    = %p\n", m);     /* 2000 (address) */
printf("q    = %d\n", q);     /* 100 */
return 0;
}

```

NOTE *The multiplication symbol `*` and the 'at address' `*` are the same character but have no relationship to each other. Similarly, the bitwise AND `&` and the 'address of' `&` are the same character but have no relationship. Context determines which meaning applies.*

POINTER EXPRESSIONS

In general, expressions involving pointers conform to the same rules as other expressions. This section examines special aspects of pointer expressions: assignments, conversions, arithmetic, and comparisons.

Pointer Assignments

You can use a pointer on the right-hand side of an assignment statement to assign its value to another pointer. When both pointers are the same type, the situation is straightforward:

```

#include <stdio.h>
int main(void)
{
    int x = 99;
    int *p1, *p2;
    p1 = &x;
    p2 = p1; /* p2 now points to the same location as p1 */

    /* print the value of x twice */
    printf("Values at p1 and p2: %d %d\n", *p1, *p2);

    /* print the address of x twice */
    printf("Addresses pointed to by p1 and p2: %p %p", p1, p2);
    return 0;
}

```

After the assignment sequence `p1 = &x`; then `p2 = p1`; — both `p1` and `p2` point to `x` and refer to the same object. Sample output:

```

Values at p1 and p2: 99 99
Addresses pointed to by p1 and p2: 0063FDF0 0063FDF0

```

```
/* Both addresses are identical — both pointers point to the same memory location */
```

NOTE *The %p format specifier causes printf() to display an address in the format used by the host computer.*

Pointer Conversions

One type of pointer can be converted into another type of pointer. There are two general categories: those that involve void * pointers, and those that don't.

Conversion Type	Rule	Cast Required?
Any pointer → void *	Permissible in C — void * is a generic pointer with unknown base type	No (automatic)
void * → any pointer	Permissible in C — used for raw memory (e.g., from malloc())	No (automatic)
Any other pointer conversion	Permissible but may create undefined behavior	Yes (explicit cast required)
Integer → pointer OR pointer → integer	Allowed but result is implementation-defined; may cause undefined behavior	Yes (explicit cast), except zero (null pointer)

The void * pointer (generic pointer):

- Used to specify a pointer whose base type is unknown
- Allows a function parameter to receive any type of pointer argument without a type mismatch
- Used to refer to raw memory — such as that returned by malloc() — when its semantics are not yet known
- No explicit cast is required to convert to or from a void * pointer

Example of dangerous pointer type mismatch — compiles without error but produces wrong result:

```
#include <stdio.h>
int main(void)
{
    double x = 100.1, y;
    int *p;
    /* Causes p (integer pointer) to point to a double */
    p = (int *) &x;
    /* Does NOT operate as expected */
    y = *p; /* attempt to assign y the value x through p */
    printf("The (incorrect) value of x is: %f", y);
    return 0;
}
```

```
/* Problem: p is an int * — only 4 bytes are transferred to y,
not the 8 bytes that make up a double.
Even though p points to a double, it still 'thinks' it points to an int.
Pointer operations are governed by the type of the POINTER, not the object. */
```

NOTE *In C++, it is illegal to convert one type of pointer into another type of pointer without an explicit type cast — this includes void * conversions. For this reason, many C programmers cast all pointer conversions so their code is also compatible with C++.*

Pointer Arithmetic

There are only two arithmetic operations that you can use on pointers: addition and subtraction.

Fundamental rule: Each time a pointer is incremented, it points to the memory location of the NEXT element of its base type. Each time it is decremented, it points to the PREVIOUS element.

Example — integer pointer p1 with initial value 2000 (assuming 2-byte integers):

```
p1++; /* p1 = 2002 (not 2001) — moves by sizeof(int) = 2 bytes */
p1--; /* p1 = 1998 — moves back by sizeof(int) = 2 bytes */

p1 = p1 + 12; /* p1 points to the 12th element beyond current position */

/* Subtracting two pointers — gives number of elements between them: */
int diff = p2 - p1; /* number of int objects between p2 and p1 */
```

How pointer increment changes vary by base type (assuming p starts at 2000):

Base Type	Size Per Element	p after p++	p after p--
char	1 byte	2001	1999
short int	2 bytes	2002	1998
int (16-bit)	2 bytes	2002	1998
int (32-bit)	4 bytes	2004	1996
double	8 bytes	2008	1992

Valid pointer arithmetic operations:

- p++ or p-- — increment / decrement (moves by sizeof(base type))
- p + n — pointer to the nth element beyond current (moves $n \times \text{sizeof}(\text{base type})$ bytes)
- p - n — pointer to the nth element before current
- p1 - p2 — number of base-type elements separating the two pointers

Invalid pointer arithmetic operations (these are prohibited):

- Multiply pointers ($p1 * p2$)
- Divide pointers ($p1 / p2$)
- Add two pointers ($p1 + p2$)
- Apply bitwise operators to pointers ($p1 \& p2$, $p1 | p2$, etc.)
- Add or subtract type float or double to/from pointers

NOTE When applied to char pointers, pointer arithmetic appears as 'normal' arithmetic because a char object is always 1 byte long in any environment. All other pointer types increase or decrease by the size of the data type they point to.

Pointer Comparisons

You can compare two pointers in a relational expression. Pointer comparisons are generally useful only when two pointers point to a common object, such as an array.

```
if(p < q) printf("p points to lower memory than q\n");
```

Example — a stack implementation using pointer comparisons to detect overflow and underflow:

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 50

void push(int i);
int pop(void);
int *tos, *p1, stack[SIZE];

int main(void)
{
    int value;
    tos = stack; /* tos points to top of stack */
    p1 = stack; /* initialize p1 */
    do {
        printf("Enter value: ");
        scanf("%d", &value);
        if(value != 0) push(value);
        else printf("value on top is %d\n", pop());
    } while(value != -1);
    return 0;
}

void push(int i)
{
    p1++;
    if(p1 == (tos + SIZE)) { /* pointer comparison: detect overflow */
```

```

    printf("Stack Overflow.\n");
    exit(1);
}
*p1 = i;
}

int pop(void)
{
    if(p1 == tos) { /* pointer comparison: detect underflow */
        printf("Stack Underflow.\n");
        exit(1);
    }
    p1--;
    return *(p1+1); /* parentheses are NECESSARY */
    /* Without parentheses: *p1+1 = value AT p1, plus 1 (wrong!) */
    /* With parentheses: *(p1+1) = value AT location p1+1 (correct) */
}

```

Stack logic explained:

- `tos` — holds the address of the top of the stack; used to prevent overflows and underflows
- `p1` — the working pointer that accesses the stack data
- `push()` — increments `p1` first, then checks: if `p1 == tos+SIZE`, the stack is full (overflow)
- `pop()` — checks: if `p1 == tos`, the stack is empty (underflow); then decrements `p1` and returns the value
- The comparison `p1 == (tos+SIZE)` and `p1 == tos` are pointer comparisons on the shared array stack

POINTERS AND ARRAYS

There is a close relationship between pointers and arrays. C provides two equivalent methods of accessing array elements: array indexing and pointer arithmetic.

```

char str[80], *p1;
p1 = str; /* p1 points to the first element of str */

/* Two equivalent ways to access the 5th element: */
str[4] /* array indexing — uses index 4 (arrays start at 0) */
*(p1+4) /* pointer arithmetic — adds 4 to p1's address */

```

C provides two methods of accessing array elements:

Method	Syntax	Notes
Array indexing	<code>str[4]</code>	Standard notation — sometimes easier to read and understand

Method	Syntax	Notes
Pointer arithmetic	*(p1+4)	Often faster; the way professional C programmers commonly write such routines

Comparison — `putstr()` function (writes a string to output one character at a time):

```
/* Version 1: Index s as an array */
void putstr(char *s)
{
    register int t;
    for(t=0; s[t]; ++t) putchar(s[t]);
}

/* Version 2: Access s as a pointer (preferred by professional C programmers) */
void putstr(char *s)
{
    while(*s) putchar(*s++);
}
/* The pointer version is more efficient and is the standard style in C.
   *s++ : dereferences s to get the character, then increments s to next position */
```

Arrays of Pointers

Pointers can be arrayed like any other data type. An array of pointers is declared by placing `*` before the array name:

```
int *x[10]; /* array of 10 integer pointers */

/* Assign the address of integer variable var to the 3rd element: */
x[2] = &var;

/* Access the VALUE of var through the pointer array: */
*x[2]
```

Passing an array of pointers to a function — use the array name without any subscripts:

```
void display_array(int *q[])
{
    int t;
    for(t=0; t<10; t++)
        printf("%d ", *q[t]);
}
/* IMPORTANT: q must be declared as an array of integer POINTERS — int *q[] */
/* NOT simply as int *q — because q is a pointer to an array of pointers, not just a pointer to int */
```

Most common use of pointer arrays — holding pointers to strings (error message tables):

```
void syntax_error(int num)
{
    static char *err[] = {
        "Cannot Open File\n",
        "Read Error\n",
        "Write Error\n",
        "Media Failure\n"
    };
    printf("%s", err[num]);
}
/* err[] holds a pointer to each string constant.
   String constants used in an expression generate a pointer to the string.
   If num=2, 'Write Error' is displayed. */
```

NOTE *The command line argument argv (used in main) is itself an array of character pointers — it works exactly like the err[] array above.*

Pointers to Functions

A function has a physical location in memory that can be assigned to a pointer. This address is the entry point of the function — the address used when the function is called. Once a pointer points to a function, the function can be called through that pointer. Function pointers also allow functions to be passed as arguments to other functions.

- The address of a function is obtained by using the function's name without any parentheses or arguments
- This is similar to the way an array's address is obtained by using only the array name without indexes

Function pointer declaration syntax:

```
int (*p)(const char *, const char *);
/* p is a pointer to a function that:
   - takes two const char * parameters
   - returns an int
   The parentheses around *p are NECESSARY for the compiler to interpret correctly */
```

Example — passing a function pointer to another function:

```
#include <stdio.h>
#include <string.h>
void check(char *a, char *b,
```

```

    int (*cmp)(const char *, const char *));

int main(void)
{
    char s1[80], s2[80];
    int (*p)(const char *, const char *); /* function pointer */
    p = strcmp; /* assign address of strcmp to p */
    printf("Enter two strings.\n");
    gets(s1); gets(s2);
    check(s1, s2, p); /* pass address of strcmp via p */
    return 0;
}

void check(char *a, char *b,
           int (*cmp)(const char *, const char *))
{
    printf("Testing for equality.\n");
    if(!(*cmp)(a, b)) printf("Equal");
    else printf("Not Equal");
}

```

Two equivalent ways to call a function through a pointer:

```

(*cmp)(a, b); /* Style 1 — original C style; clearly shows cmp is a function pointer */
cmp(a, b);    /* Style 2 — simpler but less obvious that cmp is a pointer */

/* You can also pass the function directly without a pointer variable: */
check(s1, s2, strcmp); /* passes strcmp's address directly — eliminates pointer variable */

```

When are function pointers useful?

- Passing different functions to a general-purpose routine (e.g., different comparison functions)
- Creating arrays of functions — an interpreter's parser can call support functions (sin, cos, I/O) by index rather than a large switch statement
- Implementing callbacks and event handlers

Expanded example — check() supports both alphabetical and numeric equality by receiving different comparison functions:

```

#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#include <string.h>
void check(char *a, char *b,
           int (*cmp)(const char *, const char *));
int compvalues(const char *a, const char *b);

int main(void)

```

```

{
    char s1[80], s2[80];
    printf("Enter two values or two strings.\n");
    gets(s1); gets(s2);
    if(isdigit(*s1)) {
        printf("Testing values for equality.\n");
        check(s1, s2, compvalues); /* numeric comparison */
    }
    else {
        printf("Testing strings for equality.\n");
        check(s1, s2, strcmp); /* string comparison */
    }
    return 0;
}

void check(char *a, char *b,
           int (*cmp)(const char *, const char *))
{
    if(!(*cmp)(a, b)) printf("Equal");
    else printf("Not Equal");
}

int compvalues(const char *a, const char *b)
{
    if(atoi(a)==atoi(b)) return 0;
    else return 1;
}

/* With compvalues: '0123' compares EQUAL to '123' (numeric equality) */
/* With strcmp: '0123' compares NOT EQUAL to '123' (string equality) */

```

MULTIPLE INDIRECTION

You can have a pointer point to another pointer that points to the target value. This situation is called multiple indirection, or pointers to pointers.

Level	Description	Access Pattern	Example Declaration
Single indirection	Pointer holds address of the target object	*p	int *p;
Double indirection	Pointer holds address of another pointer, which holds address of target	**p	int **p;
Triple indirection	Three levels of pointer nesting — rarely needed	***p	int ***p;

How single and double indirection differ:

- Normal pointer (single): value of pointer = address of the object containing the desired value
- Pointer to pointer (double): first pointer = address of second pointer; second pointer = address of the target object

NOTE Do NOT confuse multiple indirection with high-level data structures such as linked lists that use pointers. These are two fundamentally different concepts. Excessive indirection (more than pointer to pointer) is difficult to follow and prone to conceptual errors.

Declaring a Pointer to a Pointer

A variable that is a pointer to a pointer must be declared with an additional asterisk in front of the variable name:

```
float **newbalance;
/* newbalance is a pointer to a FLOAT POINTER */
/* It is NOT a pointer to a float */

/* General pattern: */
int *p; /* pointer to int */
int **q; /* pointer to a pointer to int */
int ***r; /* pointer to a pointer to a pointer to int */
```

Accessing the Target Value Through Double Indirection

To access the target value indirectly pointed to by a pointer to a pointer, you must apply the * operator TWICE:

```
#include <stdio.h>
int main(void)
{
    int x, *p, **q;
    x = 10;
    p = &x; /* p points to x */
    q = &p; /* q points to p */
    printf("%d", **q); /* print the value of x */
    return 0;
}
/* Output: 10 */
/* **q: first * dereferences q → gets p (address of x) */
/* second * dereferences p → gets x (value 10) */
```

Step-by-step trace of the double indirection:

Expression	Value	Meaning
Q	address of p	q holds the address of the pointer p
*q	address of x (= p)	dereferencing q gives the value of p, which is the address of x
**q	10 (= x)	dereferencing *q (= p) gives the value of x, which is 10

INITIALIZING POINTERS

After a nonstatic, local pointer is declared but before it has been assigned a value, it contains an unknown (garbage) value. If you try to use it before assigning a valid address, you will probably crash your program — and possibly your computer's operating system. This is a very nasty type of error.

Pointer Type	Initial Value (before explicit assignment)	Safe to Use?
Local (nonstatic) pointer	Unknown / garbage value	NO — must be assigned a valid address first
Global pointer	Automatically initialized to null (0)	Only safe if null is checked before use
Static local pointer	Automatically initialized to null (0)	Only safe if null is checked before use

The Null Pointer Convention

There is an important convention that most C programmers follow: A pointer that does not currently point to a valid memory location is given the value null (zero). Null is used because C guarantees that no object will exist at the null address. Thus, any pointer that is null implies that it points to nothing and should not be used.

```
/* Two equivalent ways to initialize a pointer to null: */
char *p = 0; /* assign integer 0 directly */
p = NULL; /* use the NULL macro (defined in <stdio.h> and other headers) */
```

Why null is important:

- C guarantees that no object exists at address zero — so a null pointer cannot accidentally point to valid data
- The use of null to indicate unused pointers is a widely followed programmer convention
- It is NOT a rule enforced by the C language — a null pointer can still be incorrectly dereferenced

Example of dangerous code — compiles without error but will crash:

```
int *p = 0;
*p = 10; /* WRONG! Dereferences null pointer → assignment to address 0 → crash */
```

NOTE *Just because a pointer has a null value, it is not necessarily 'safe.' The use of null to indicate unused pointers is a convention, not a language enforcement mechanism. Always check for null before dereferencing.*

Using Null Pointers as Sentinels

Because a null pointer is assumed to be unused, it can be used to mark the end of a pointer array. A routine that accesses such an array knows it has reached the end when it encounters null — making the code easier and more efficient.

Example — `search()` function: given a list of names, determines whether a specified name is in the list:

```
#include <stdio.h>
#include <string.h>
int search(char *p[], char *name);

char *names[] = {
    "Herb",
    "Rex",
    "Dennis",
    "John",
    NULL /* null pointer constant marks end of list */
};

int main(void)
{
    if(search(names, "Dennis") != -1)
        printf("Dennis is in list.\n");
    if(search(names, "Bill") == -1)
        printf("Bill not found.\n");
    return 0;
}

/* Look up a name — returns index if found, -1 if not found */
int search(char *p[], char *name)
{
    register int t;
    for(t=0; p[t]; ++t) /* loop ends when p[t] is null */
        if(!strcmp(p[t], name)) return t;
    return -1; /* not found */
}

/* The for loop condition 'p[t]' is false when p[t] is null (zero) */
/* This is how the null sentinel naturally terminates the search */
```

Initializing `char *` Pointers to String Constants

C programmers commonly initialize `char *` pointers to point to string constants. When a string constant is used in an expression (such as an initialization), it produces a pointer to the start of the string.

```
char *p = "hello world";  
/* p is a POINTER, not an array. */  
/* The compiler stores 'hello world' in a string table and initializes p */  
/* to point to the first character 'h' in that string table. */
```

How C handles string constants:

- The C compiler creates a string table that stores the string constants used by the program
- The declaration places the address of "hello world" as stored in the string table into the pointer `p`
- Throughout the program, `p` can be used like any other string — it can be passed to `puts()`, `printf()`, `strlen()`, etc.

Example — using a `char *` string pointer to print a string forward and backwards:

```
#include <stdio.h>  
#include <string.h>  
char *p = "hello world";  
int main(void)  
{  
    register int t;  
    /* print the string forward */  
    printf(p);  
    /* print the string backwards */  
    for(t=strlen(p)-1; t>=0; t--)  
        printf("%c", p[t]);  
    return 0;  
}
```

Comparison of `char *` pointer vs `char` array for strings:

Feature	<code>char *p = "hello";</code>	<code>char p[] = "hello";</code>
Type	Pointer to a string in the string table	Array containing the characters
Memory	String stored in string table; <code>p</code> holds its address	Characters stored directly in <code>p[0]</code> through <code>p[5]</code>
Can change what <code>p</code> points to?	Yes: <code>p = "another string";</code>	No: <code>p</code> is a fixed array, cannot be reassigned

Feature	<code>char *p = "hello";</code>	<code>char p[] = "hello";</code>
Modify string contents?	Undefined behavior (string table may be read-only)	Yes: <code>p[0] = 'H'</code> ; is safe
Usage with string functions	Works the same — can pass to <code>strlen()</code> , <code>strcmp()</code> , etc.	Works the same

DYNAMIC ALLOCATION — `malloc()` AND `free()`

C's dynamic allocation functions allow memory to be allocated and freed at run time from a region of memory called the heap. These functions are declared in `<stdlib.h>`.

`malloc()` — Allocate Memory

The `malloc()` function allocates a block of contiguous memory from the heap and returns a pointer to the first byte of the allocated region:

```
void *malloc(size_t number_of_bytes);
```

- Returns a `void *` pointer — automatically converted to the appropriate pointer type on assignment (in C, but NOT in C++ where an explicit cast is needed)
- Returns null if there is not enough available memory to satisfy the request
- Always check the returned pointer for null before using it

```
char *p;
p = malloc(1000); /* allocate 1000 bytes; p points to first byte */

/* Use sizeof for portability: */
int *ip;
ip = malloc(50 * sizeof(int)); /* space for 50 integers */

/* Always test for null: */
p = malloc(100);
if(!p) {
    printf("Out of memory.\n");
    exit(1);
}
```

`free()` — Release Memory

The `free()` function returns previously allocated memory to the heap so it can be reused by subsequent calls to `malloc()`:

```
void free(void *p);
```

- `p` must be a pointer to memory previously allocated using `malloc()`
- It is critical NEVER to call `free()` with an invalid argument — this will damage the allocation system
- After freeing, the pointer should not be used without first reassigning it a valid address

Dynamically Allocated Arrays

Since any pointer can be indexed as if it were an array, you can use `malloc()` to create dynamically allocated arrays and access them using standard array indexing notation.

Example — dynamically allocate space for a string, input it, then print it backwards:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
int main(void)
{
    char *s;
    register int t;
    s = malloc(80);
    if(!s) {
        printf("Memory request failed.\n");
        exit(1);
    }
    gets(s);
    for(t=strlen(s)-1; t>=0; t--) putchar(s[t]);
    free(s);
    return 0;
}
/* s is used like any array — passed to gets(), indexed with s[t], freed with free() */
```

Dynamically allocating a multidimensional array — declare a pointer specifying all but the leftmost dimension:

```
/* Build a 4×10 table of powers (1-10 raised to 1st, 2nd, 3rd, 4th power) */
int (*p)[10]; /* p is a pointer to an array of 10 integers */
/* Parentheses around *p are NECESSARY */

p = malloc(40 * sizeof(int)); /* allocate 4 rows × 10 columns × 4 bytes */
if(!p) { printf("Memory request failed.\n"); exit(1); }

/* p can now be indexed as a 2D array: */
for(j=1; j<11; j++)
    for(i=1; i<5; i++) p[i-1][j-1] = pwr(j, i);
```

```
/* For C/C++ compatibility, cast the malloc return: */
p = (int (*)(10)) malloc(40 * sizeof(int));
```

NOTE When *p* is declared as `int (*p)[10]`, its base type is a 10-int array. When *p* is incremented, it points to the start of the next 10 integers. When decremented, it points to the previous 10. This allows *p* to be used as a pointer to a 2D array with 10 elements per row.

restrict-QUALIFIED POINTERS (C99)

The C99 standard adds a new type qualifier that applies only to pointers: `restrict`.

Aspect	Description
What it means	A restrict-qualified pointer is initially the only means by which the object it points to is accessed
Second pointer rule	Access by another pointer is only permitted if the second pointer is based on the first (derived from it)
Primary uses	Function parameters, or pointers to memory allocated via <code>malloc()</code>
Benefit to compiler	The compiler can assume restrict-qualified pointer parameters point to non-overlapping objects, enabling better optimization
Effect on semantics	<code>restrict</code> does NOT change the semantics of a program — it is purely a hint to the optimizer
Where described in full	Part Two of the textbook

PROBLEMS WITH POINTERS

Nothing will get you into more trouble than a wild pointer! Pointers give you tremendous power, but when used incorrectly or when they contain the wrong value, they can be very difficult bugs to find.

Why pointer errors are so hard to find:

- The pointer itself is not the problem — trouble starts when you access an object through a bad pointer
- Reading through a bad pointer gives garbage values, causing the program to malfunction
- Writing through a bad pointer overwrites other code or data — often causing crashes much later in execution
- The problem may not appear until late in program execution, making you look for the bug in the wrong place
- There may be little evidence that the pointer was the original cause of the problem

The Classic Error: Uninitialized Pointer

The most common pointer error is using a pointer before assigning it a valid value:

```
/* This program is WRONG */
int main(void)
{
    int x, *p;
    x = 10;
    *p = x; /* ERROR: p has never been initialized! */
    return 0;
}
/* p contains an unknown (garbage) value. The assignment *p = x
writes the value 10 to some unknown memory location.
This may corrupt data, code, or the operating system. */
```

Why this error is especially dangerous:

- When the program is small, p may accidentally contain a 'safe' address not in your code or data area — the bug goes unnoticed
- As the program grows, the probability increases that p points to something vital
- Eventually, the program stops working — often far from the actual cause
- Most compilers issue a warning for this specific case, but the program still compiles

Common Pointer Errors Summary

Error Type	Description	Prevention
Uninitialized pointer	Using a pointer before assigning it a valid address	Always initialize pointers to NULL or a valid address immediately upon declaration
Null pointer dereference	Dereferencing a pointer that holds NULL (0)	Always check: if(p != NULL) before using *p
Out-of-bounds access	Pointer arithmetic moves the pointer outside the valid range of an array	Keep track of array bounds; never exceed them
Dangling pointer	Using a pointer after the memory it points to has been freed	Set pointer to NULL immediately after calling free()
Type mismatch	Casting a pointer to the wrong type and then dereferencing it	Always use the correct base type; avoid unnecessary type casts
Double free	Calling free() on the same pointer twice	Set pointer to NULL after free(); check for NULL before freeing

CHAPTER 5 — QUICK REFERENCE SUMMARY

Pointer Declarations

Declaration	Meaning
<code>int *p;</code>	p is a pointer to an integer
<code>double *p;</code>	p is a pointer to a double
<code>char *p;</code>	p is a pointer to a character
<code>void *p;</code>	p is a generic pointer (unknown base type)
<code>int **p;</code>	p is a pointer to a pointer to an integer
<code>int *x[10];</code>	x is an array of 10 integer pointers
<code>int (*p)[10];</code>	p is a pointer to an array of 10 integers (used for 2D dynamic arrays)
<code>int (*p)(char *, char *);</code>	p is a pointer to a function taking two char * args returning int

Pointer Operators

Operator	Name	Effect	Example
<code>&</code>	Address-of	Returns the memory address of the operand	<code>m = &count;</code>
<code>*</code>	Dereference	Returns the value at the address	<code>q = *m;</code>
<code>++</code>	Increment	Moves pointer to next element of its base type	<code>p++;</code>
<code>--</code>	Decrement	Moves pointer to previous element of its base type	<code>p--;</code>
<code>+ n</code>	Add integer	Moves pointer n elements forward	<code>p = p + 5;</code>
<code>- n</code>	Subtract integer	Moves pointer n elements back	<code>p = p - 3;</code>
<code>p1 - p2</code>	Pointer subtraction	Number of base-type elements between two pointers	<code>int n = p2 - p1;</code>

Pointer Arithmetic — Valid vs Invalid

Operation	Valid?	Notes
<code>p++</code> or <code>p--</code>	YES	Moves by <code>sizeof(base type)</code> bytes, not 1 byte
<code>p + n</code> or <code>p - n</code>	YES	Moves <code>n</code> elements forward/backward
<code>p1 - p2</code>	YES	Number of base-type elements between <code>p1</code> and <code>p2</code>
<code>p1 + p2</code>	NO	Cannot add two pointers
<code>p * n</code> or <code>p / n</code>	NO	Cannot multiply or divide a pointer
<code>p & mask</code> or <code>p mask</code>	NO	Cannot apply bitwise ops to pointers
<code>p + 1.5</code> (float/double)	NO	Cannot add float/double to a pointer

Pointer Initialization Rules

Rule	Detail
Local pointer — uninitialized	Contains garbage; MUST be assigned a valid address before use
Global / static local pointer	Automatically initialized to null (0)
Null convention	Assign NULL or 0 to a pointer when it doesn't point to a valid object
Check before use	Always test: <code>if(p != NULL)</code> before dereferencing a pointer
Set NULL after <code>free()</code>	After <code>free(p)</code> , set <code>p = NULL</code> to prevent dangling pointer usage
NULL macro	Defined in <code><stdio.h></code> and other headers as a null pointer constant

Multiple Indirection Quick Reference

Type	Declaration	Dereference to get value	Description
Single indirection	<code>int *p;</code>	<code>*p</code>	<code>p</code> holds address of an <code>int</code>
Double indirection	<code>int **p;</code>	<code>**p</code>	<code>p</code> holds address of an <code>int</code> pointer
Triple indirection	<code>int ***p;</code>	<code>***p</code>	<code>p</code> holds address of a pointer to an <code>int</code> pointer

— End of Chapter 5 Notes —